Manipulación, validación y administración en PostgreSQL

Breve descripción:

Este componente formativo capacita al aprendiz en la gestión avanzada del ciclo de vida de los datos y el aseguramiento del servidor. Se profundiza en la escritura de consultas (DML), uniones relacionales y programación procedural nativa (PL/pgSQL), complementándose con la implementación de directivas de seguridad, auditoría transaccional, optimización de rendimiento y protocolos de respaldo.

Septiembre 2026

Tabla de contenido

Introducción 4

1. Operaciones de modificación de datos y consultas esenciales (CRUD) 5

1.1. Operaciones esenciales del ciclo de vida de los datos 5

1.2. Consultas básicas y procesamiento de datos 8

2. Filtrado, agregación y consolidación de datos (joins) 14

2.1. Consultas y agrupamientos 14

2.2. Relaciones y uniones de datos (Joins) 16

3. Lógica programable en el servidor (PL/pgSQL) 26

3.1. Programación procedural en PostgreSQL 26

3.2. Control de errores y robustez del código 30

4. Automatización reactiva y estructuras de abstracción de datos 37

4.1. Automatización del negocio mediante disparadores 37

4.2. Abstracción lógica y optimización de consultas 40

5. Perímetros de seguridad, control de acceso granular y continuidad del negocio 48

5.1. Directivas de seguridad perimetral y autenticación en el servidor 48

5.2. Administración de identidades corporativas y matriz de privilegios privados 51

5.3. Protocolos de respaldo 53

Síntesis 57

Glosario 58

Referencias bibliográficas 60

Créditos 61

Introducción

Una vez estructurada la arquitectura física de un repositorio relacional, el verdadero valor de los sistemas de información radica en la capacidad de explotar, automatizar y proteger los datos almacenados. Este componente formativo se introduce en las fases de la ingeniería de datos en PostgreSQL, proporcionando las herramientas técnicas para interactuar de forma dinámica con el motor y transformar los datos en bruto en conocimiento estratégico.

El propósito central de este material es desarrollar la competencia para manipular el flujo informativo mediante el uso del lenguaje SQL y la programación del lado del servidor. A lo largo del componente, se abordará desde la construcción de consultas e integraciones de tablas (Joins), hasta la creación de lógica procedural mediante funciones y disparadores (Triggers) que automaticen las reglas de negocio de manera reactiva e indestructible.

Para garantizar un perfil técnico integral, el proceso de aprendizaje abarca las responsabilidades de la administración de servidores. El aprendiz asimilará los protocolos de seguridad perimetral y granular, el monitoreo a través de bitácoras de auditoría (Logs) y la ejecución de copias de seguridad, asegurando que la solución informática permanezca optimizada, blindada ante vulnerabilidades y preparada para esquemas de alta disponibilidad.

1. Operaciones de modificación de datos y consultas esenciales (CRUD)

Una vez que la infraestructura física y el esquema de tablas han sido desplegados con éxito en el servidor, el ciclo de vida de la información entra en su fase más dinámica e interactiva. La capacidad de poblar, corregir y depurar los registros dentro de un repositorio digital constituye la competencia procedimental básica que conecta la base de datos con la lógica de negocio de cualquier aplicación informática.

En esta unidad temática se abordarán las sentencias fundamentales del Lenguaje de Manipulación de Datos (DML) bajo el estándar de PostgreSQL. A partir de ello, se comprenderán las operaciones esenciales de inserción, actualización y borrado condicional, así como los mecanismos básicos de filtrado que garantizan la extracción precisa y oportuna de los activos de información de una organización.

1.1. Operaciones esenciales del ciclo de vida de los datos

El mantenimiento y la persistencia de la información exigen el dominio de las operaciones conocidas mediante el acrónimo CRUD (`Create`, `Read`, `Update` y `Delete`; Crear, Leer, Actualizar y Borrar). En una solución informática concurrente, estas instrucciones alteran directamente los bloques físicos de almacenamiento y activan los mecanismos internos de control transaccional del motor de base de datos para preservar las propiedades ACID (Aislamiento, Consistencia, Integridad y Durabilidad).

Sentencias de inserción masiva y selectiva de registros (INSERT INTO)

La creación de nuevos registros se realiza mediante la instrucción INSERT INTO, la cual comunica al proceso del servidor la intención de añadir filas lógicas a una relación física específica. La sintaxis estándar exige declarar explícitamente la tabla de destino, la

lista de atributos que se desean poblar y los valores correspondientes, organizados por tuplas (Silberschatz; Korth, y Sudarshan, 2002).

PostgreSQL amplía las capacidades del estándar ANSI SQL al permitir la inserción masiva en una sola sentencia mediante la separación de tuplas por comas. Este mecanismo reduce significativamente el tráfico de red y optimiza el tiempo de procesamiento, ya que abre una única transacción en lugar de ejecutar múltiples peticiones independientes.

De manera complementaria, el motor incorpora la cláusula avanzada RETURNING, la cual devuelve inmediatamente los valores generados automáticamente por el servidor, como llaves primarias de tipo SERIAL o valores definidos por defecto. Gracias a esta funcionalidad, la aplicación evita realizar una consulta adicional para recuperar dichos datos.

La siguiente sentencia realiza una inserción masiva de registros y utiliza la cláusula RETURNING para recuperar automáticamente los identificadores generados por PostgreSQL:

Transcripción de código. Inserción masiva optimizada con cláusula de retorno en PostgreSQL
INSERT INTO ejemplo_sena (codigo_modulo, estado) VALUES ('MOD-001', TRUE), ('MOD-002', FALSE),

```
 ('MOD-003', TRUE)

RETURNING id_registro;
```

Modificación y actualización condicional de datos (UPDATE)

Un principio fundamental de seguridad consiste en comprender que la sentencia UPDATE opera, por defecto, sobre la totalidad de las filas de la tabla (Elmasri y Navathe, 2021). Por esta razón, resulta indispensable incorporar la cláusula WHERE, la cual delimita de forma precisa el subconjunto de registros que será actualizado.

Desde la perspectiva interna del motor PostgreSQL, y debido al mecanismo de control de concurrencia multiversión (MVCC), la ejecución de un UPDATE no sobrescribe los datos originales. En su lugar, marca la versión anterior de la fila como obsoleta y genera una nueva versión del registro, dejando al proceso autovacuum la responsabilidad de efectuar posteriormente la limpieza física correspondiente.

Modificación y actualización condicional de datos (UPDATE)

Cuando la información pierde vigencia operativa, PostgreSQL dispone de dos comandos con finalidades similares, pero con comportamientos internos y efectos sobre el rendimiento claramente diferenciados.

A continuación, se resumen las principales características de cada uno de ellos:

- **DELETE.** Elimina registros de forma selectiva mediante la cláusula WHERE. Opera bajo el esquema MVCC, por lo que genera registros muertos que permanecen almacenados hasta que se ejecuta el proceso de limpieza física (autovacuum) (PostgreSQL 16.0 Documentation, 2026). Es una operación

completamente transaccional y permite revertir los cambios mediante ROLLBACK.

- **TRUNCATE.** Elimina de forma inmediata la totalidad de las filas de una tabla. No recorre los registros individualmente ni genera bitácoras de tuplas muertas; en su lugar, desasigna directamente las páginas de almacenamiento a nivel del sistema de archivos del sistema operativo. Al omitir el procesamiento transaccional granular, ofrece un rendimiento significativamente superior, aunque requiere privilegios administrativos debido a su impacto sobre la estructura física de la tabla.

La elección entre DELETE y TRUNCATE dependerá del objetivo de la operación, del volumen de información involucrado y del nivel de control transaccional requerido durante el proceso de administración de la base de datos.

1.2. Consultas básicas y procesamiento de datos

La extracción selectiva de la información constituye una de las operaciones más frecuentes en los entornos de producción. Las aplicaciones informáticas requieren respuestas inmediatas para poblar interfaces de usuario, validar procesos de autenticación y soportar la lógica de negocio. En este contexto, la sentencia SELECT representa el mecanismo central para consultar información, ya que implementa los operadores matemáticos de proyección y selección del modelo relacional.

Cláusulas de restricción fundamental (WHERE), operadores lógicos y relacionales

La cláusula #WHERE constituye el principal mecanismo de filtrado de las consultas. Cuando el proceso hijo del servidor ejecuta un escaneo sobre una tabla,

evalúa cada tupla frente a la expresión condicional definida en esta cláusula. Únicamente aquellas cuyo resultado lógico sea TRUE serán incluidas en el conjunto de resultados enviado al cliente.

Para construir criterios de búsqueda, es posible combinar operadores relacionales estándar (=, <>, >, <, >=, <=) con operadores lógicos (AND, OR y NOT). Esta combinación permite expresar reglas de negocio cada vez más específicas. Adicionalmente, PostgreSQL incorpora operadores especializados para realizar búsquedas sobre datos de tipo texto:

- **LIKE.** Busca coincidencias de texto respetando la diferencia entre mayúsculas y minúsculas. Utiliza los comodines % (cualquier cantidad de caracteres) y _ (un único carácter).
- **ILIKE.** Realiza búsquedas sin distinguir entre mayúsculas y minúsculas. También emplea los comodines % y _, facilitando consultas más flexibles sobre información textual.

Estos operadores permiten localizar fragmentos de texto dentro de los campos alfanuméricos almacenados en la base de datos.

Evaluación de nulos (IS NULL) y criterios de ordenación extendida (ORDER BY)

Un error frecuente al consultar bases de datos consiste en intentar evaluar valores nulos mediante el operador de igualdad convencional, por ejemplo:

columna = NULL

Esta comparación nunca produce el resultado esperado, ya que NULL representa la ausencia de información o un valor desconocido. En consecuencia, cualquier comparación directa se evalúa bajo la lógica trivalente del modelo relacional (Elmasri y Navathe, 2021).

Para realizar esta validación correctamente, PostgreSQL dispone de los operadores unarios IS NULL e IS NOT NULL, diseñados específicamente para identificar la presencia o ausencia de valores nulos.

Por otra parte, los motores de bases de datos relacionales no garantizan un orden físico determinado al devolver los registros de una consulta. Cuando una aplicación necesita presentar información organizada cronológica, numérica o alfabéticamente, debe incorporar explícitamente la cláusula ORDER BY. A continuación, se presentan las directivas utilizadas para definir el criterio de ordenamiento:

- **ASC**. Organiza los registros en orden ascendente. Corresponde al comportamiento predeterminado cuando no se especifica una dirección de ordenamiento.
- **DESC**. Organiza los registros en orden descendente.

El uso de ORDER BY permite obtener resultados consistentes y facilita la presentación de la información en aplicaciones y sistemas gestores de bases de datos.

Caso de uso: El sistema de gestión de recaudo de compañía de transporte (fase de operaciones CRUD)

Una empresa de transporte cuenta con la base de datos física implementada en la nube. A partir de esta infraestructura, el administrador de bases de datos (DBA)

desarrolla el módulo del backend encargado de procesar la manipulación diaria de los despachos y las consultas relacionadas con el estado operativo de la flota.

Con el propósito de garantizar una interacción segura con el servidor PostgreSQL, el DBA diseña y valida el siguiente conjunto de sentencias del Lenguaje de Manipulación de Datos (DML). El siguiente bloque de sentencias ejemplifica las principales operaciones CRUD en PostgreSQL, incluyendo la inserción de registros, la actualización condicional y la consulta de información mediante criterios de filtrado y ordenamiento:

Transcripción de código. Principales operaciones CRUD en PostgreSQL

```
-- 1. Poblado inicial de la flota mediante inserción masiva optimizada
INSERT INTO vehiculos (placa, modelo, capacidad_pasajeros)
VALUES
  ('TSV123', 2018, 42),
  ('TSV456', 2022, 38),
  ('TSV789', 2015, 40)
RETURNING placa;
-- 2. Registro condicional de una asignación de turno para el día actual
INSERT INTO conductores (cedula, nombre_completo, numero_licencia,
numero_telefono)
VALUES ('10203040', 'Jaime Tejada', 'LIC-9988', '3001234567');
```

```
INSERT INTO asignacion_turnos (placa_vehiculo, cedula_conductor,
valor_base_recaudo)
VALUES ('TSV123', '10203040', 150000);
-- 3. Actualización selectiva de seguridad: ajuste del valor de recaudo por corrección
de planilla
UPDATE asignacion_turnos
SET valor_base_recaudo = 155000
WHERE placa_vehiculo = 'TSV123'
  AND fecha_turno = CURRENT_DATE;
-- 4. Consulta de auditoría: Extraer vehículos modernos con capacidad crítica
SELECT placa, capacidad_pasajeros
FROM vehiculos
WHERE modelo >= 2018
  AND capacidad_pasajeros > 40
ORDER BY capacidad_pasajeros DESC;
```

Una vez ejecutadas las sentencias, es posible analizar el comportamiento interno del motor de base de datos y la forma en que este garantiza la integridad de la información:

- **Garantía de la consistencia referencial.** Si se intenta ejecutar una sentencia INSERT INTO asignacion_turnos utilizando una placa inexistente, por ejemplo,

'XYZ999', el proceso hijo de PostgreSQL abortará inmediatamente la transacción y generará una excepción por violación de clave foránea (FOREIGN KEY constraint). Este comportamiento demuestra cómo el motor protege automáticamente la consistencia de los datos frente a registros inválidos provenientes de la aplicación frontend.

- **Mitigación de riesgos en las modificaciones.** Al incorporar las restricciones placa_vehiculo y fecha_turno dentro de la cláusula WHERE de la sentencia UPDATE, el desarrollador limita el alcance de la modificación únicamente al turno correspondiente del vehículo seleccionado. Esta práctica evita actualizaciones masivas no deseadas y preserva la integridad del histórico operativo y contable de la empresa de transporte.

2. Filtrado, agregación y consolidación de datos (joins)

Los datos rara vez se consumen de forma aislada o en estructuras planas. La normalización de las bases de datos relacionales, indispensable para evitar la redundancia y garantizar la integridad referencial, exige distribuir la información de manera óptima en múltiples relaciones interconectadas. En consecuencia, la capacidad de un desarrollador para unificar, resumir y transformar esos fragmentos de información en reportes consolidados e indicadores de desempeño constituye una competencia esencial para apoyar la toma de decisiones gerenciales.

2.1. Consultas y agrupamientos

Las aplicaciones informáticas que interactúan con bases de datos corporativas, capaces de almacenar millones de tuplas, requieren estrategias que permitan recuperar únicamente la información necesaria. Extraer todos los registros de forma indiscriminada incrementa el tráfico de red y eleva el consumo de memoria del cliente, afectando el rendimiento general del sistema.

De igual manera, la construcción de cuadros de mando (dashboards) ejecutivos demanda consolidar grandes volúmenes de transacciones individuales en indicadores de desempeño que faciliten el análisis del negocio.

Optimización de paginación de resultados (LIMIT, OFFSET)

La transferencia de registros innecesarios constituye uno de los errores de arquitectura más frecuentes en el desarrollo de aplicaciones. Para reducir este impacto, PostgreSQL incorpora las cláusulas LIMIT y OFFSET, las cuales constituyen la base técnica de la paginación de resultados. A continuación, se presentan las características de cada una de estas directivas:

- **LIMIT**. Restringe el número máximo de filas que el servidor devuelve en el conjunto de resultados.
- **OFFSET**. Indica la cantidad de filas que el motor debe omitir antes de comenzar a retornar los registros solicitados.

Desde la perspectiva de la ingeniería de rendimiento, el uso de valores elevados en OFFSET sobre relaciones con millones de filas puede disminuir la eficiencia de las consultas, debido a que el motor debe recorrer internamente los registros omitidos antes de entregar los resultados.

Por esta razón, se recomienda combinar siempre LIMIT y OFFSET con la cláusula ORDER BY, garantizando un orden determinista y consistente entre las diferentes páginas de resultados presentadas en las interfaces de usuario.

Optimización de paginación de resultados (LIMIT, OFFSET)

Las funciones de agregación SUM, AVG, COUNT, MAX y MIN realizan operaciones matemáticas sobre un conjunto de valores pertenecientes a una misma columna y producen un único resultado consolidado de interés estadístico.

Cuando estos cálculos deben efectuarse por categorías específicas del negocio, se utiliza la cláusula GROUP BY, la cual agrupa lógicamente las tuplas que comparten los mismos valores en una o varias columnas, aplicando la función de agregación de manera independiente a cada grupo.

Uno de los aspectos fundamentales del lenguaje SQL consiste en distinguir el momento en que actúan las cláusulas WHERE y bHAVING durante el procesamiento de una consulta. A continuación, se presenta la diferencia entre ambas cláusulas.

- **WHERE**. Filtra las filas individuales antes de que se ejecute el proceso de agrupamiento realizado por GROUP BY.
- **HAVING**. Filtra los resultados obtenidos después de que GROUP BY consolida la información, permitiendo aplicar condiciones sobre funciones de agregación. Por ejemplo, restringir el reporte únicamente a las categorías cuyo promedio de ventas supera un valor determinado.

Comprender la diferencia entre WHERE y HAVING permite construir consultas más eficientes y obtener reportes consolidados que respondan con precisión a las necesidades de análisis de la organización.

2.2. Relaciones y uniones de datos (Joins)

La consolidación de esquemas normalizados constituye el núcleo operativo de las consultas relacionales. El almacenamiento físico distribuye los atributos de las entidades en diferentes relaciones para preservar la consistencia de la información y evitar redundancias; por ejemplo, impide que los datos de un cliente se repitan en cada factura. Sin embargo, las aplicaciones y los usuarios requieren consultar esa información de forma integrada.

Las operaciones de unión, conocidas como Joins, permiten satisfacer esta necesidad al combinar filas de dos o más relaciones mediante una columna común, generalmente una clave foránea, que referencia una clave primaria.

Uniones internas (INNER JOIN) para la coincidencia exacta de registros en el modelo físico

La operación INNER JOIN constituye el mecanismo de unión más utilizado en la ingeniería de datos. Su comportamiento consiste en comparar las relaciones

involucradas y devolver únicamente aquellas tuplas que cumplen la condición de igualdad especificada en la cláusula ON (Elmasri y Navathe, 2021).

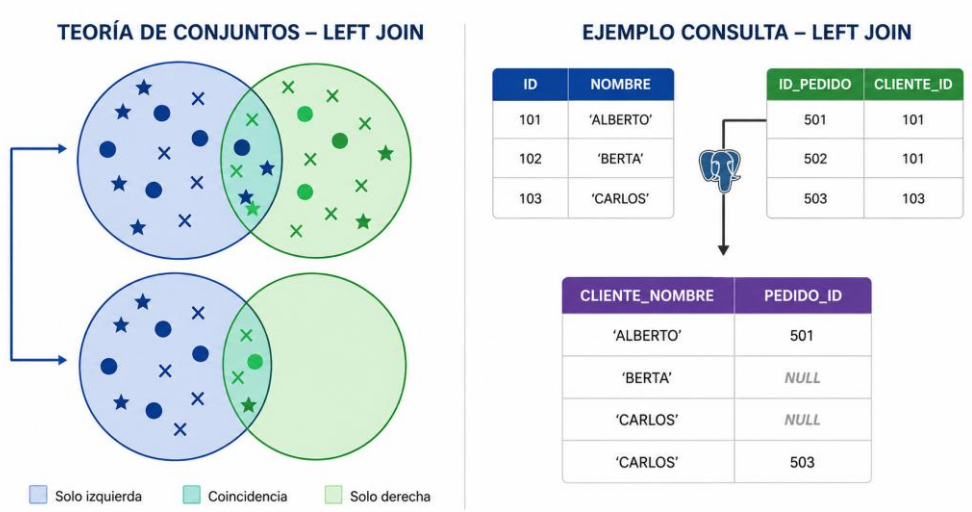
Cuando un registro de la primera relación no posee una correspondencia en la segunda, dicho registro se excluye completamente del conjunto de resultados. Por esta razón, INNER JOIN se utiliza en procesos donde la existencia de la relación entre las entidades es indispensable para la lógica de negocio de la aplicación.

Uniones externas (LEFT, RIGHT, FULL OUTER JOIN) para la preservación de tuplas no coincidentes

En determinados escenarios es necesario conservar registros que no poseen correspondencia en otra relación, por ejemplo, durante procesos de auditoría, conciliación o análisis de información incompleta. Para estos casos, PostgreSQL incorpora las uniones externas (Outer Joins), cuyo comportamiento varía según la relación cuyos registros se desee preservar. A continuación, se presentan las principales características de cada tipo de unión externa:

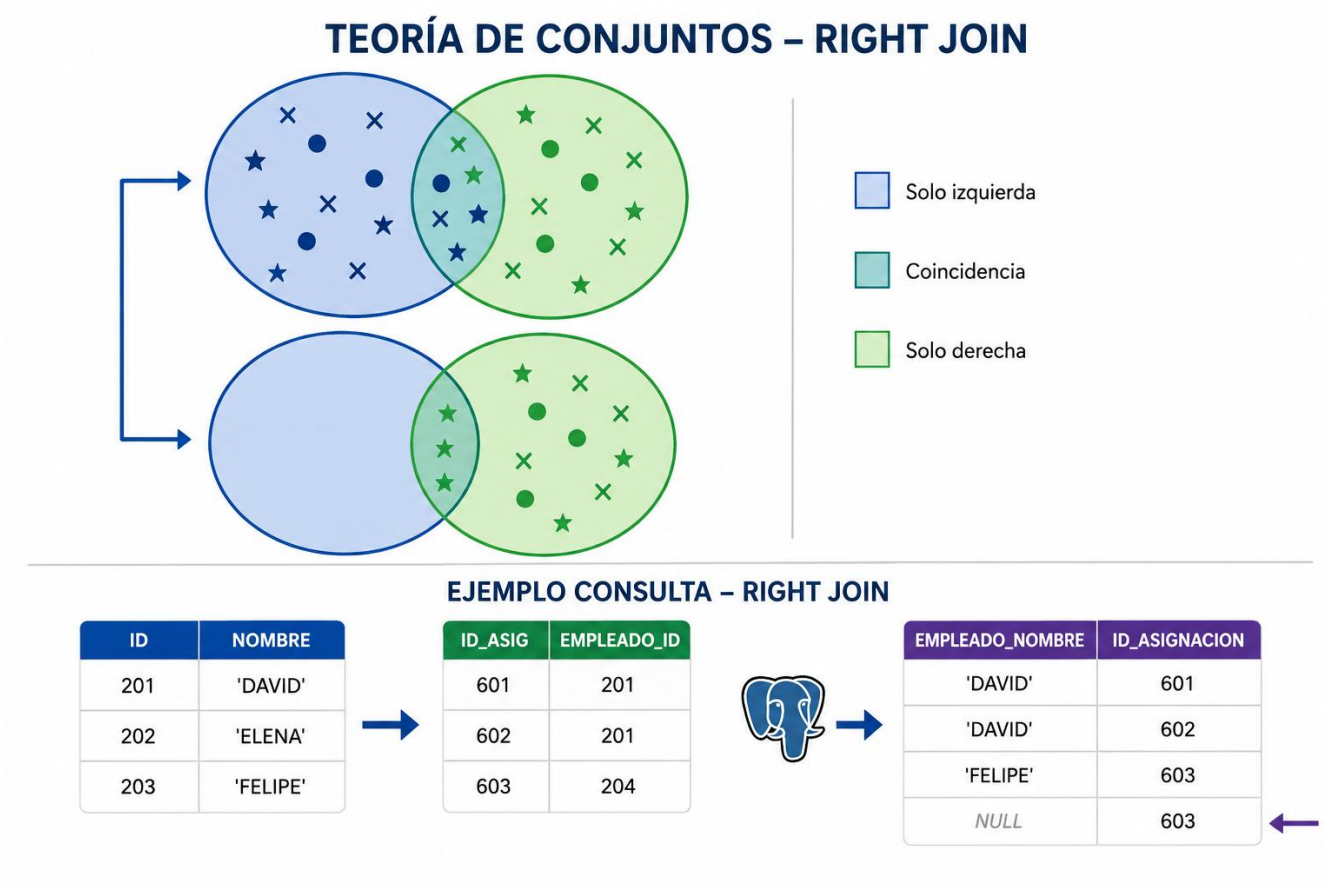
- **LEFT OUTER JOIN (LEFT JOIN).** Devuelve todos los registros de la relación ubicada a la izquierda de la sentencia, junto con las coincidencias encontradas en la relación de la derecha. Cuando no existe correspondencia, las columnas de la relación derecha se completan con valores NULL. Este tipo de unión resulta útil para identificar, por ejemplo, clientes que nunca han realizado una compra.

Figura 1. Dinámica de left join



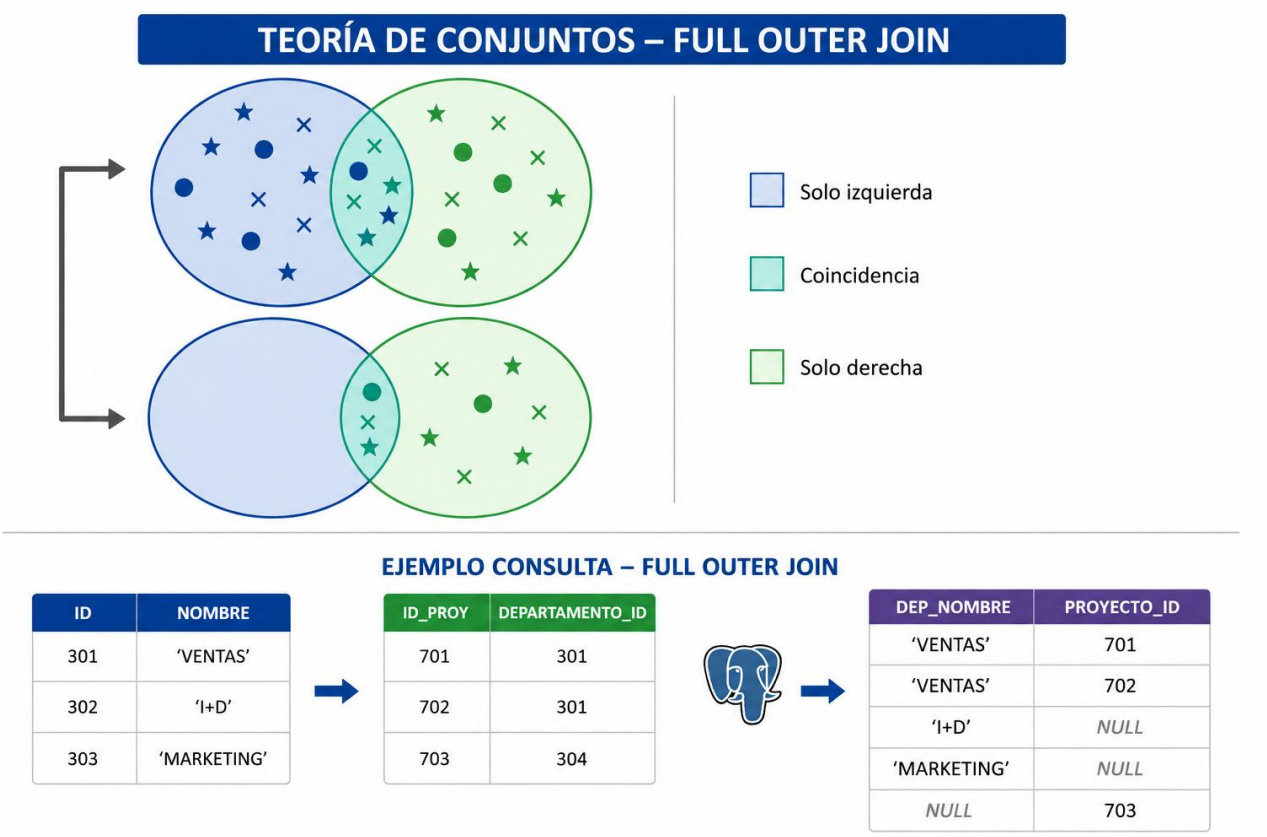
- **RIGHT OUTER JOIN (RIGHT JOIN).** Conserva todos los registros de la relación ubicada a la derecha de la sentencia y agrega la información correspondiente de la relación izquierda cuando existe coincidencia. En ausencia de ella, las columnas de la relación izquierda se completan con NULL. En la práctica, este comportamiento puede obtenerse invirtiendo el orden de las relaciones en un LEFT JOIN.

Figura 2. Dinámica de right join



- **FULL OUTER JOIN.** Representa la unión más inclusiva. Devuelve todos los registros de ambas relaciones, establece las coincidencias cuando existen y completa con NULL los atributos correspondientes cuando una de las relaciones carece de registros asociados.

Figura 3. Dinámica de full outer join



Ejecución de comandos múltiples independientes y subconsultas correlacionadas complejas

En escenarios de análisis avanzado, es frecuente que el resultado de una consulta sirva como criterio de filtrado o como fuente de información para otra consulta de nivel superior. Esta funcionalidad se implementa mediante las subconsultas (Subqueries), las cuales permiten construir consultas jerárquicas para resolver requerimientos complejos de procesamiento de datos. Las subconsultas pueden clasificarse en dos tipos principales.

- **Subconsulta independiente.** Se ejecuta una única vez y entrega un valor o un conjunto de valores que la consulta principal utiliza como criterio de evaluación.
- **Subconsulta correlacionada.** Hace referencia a una o más columnas de la consulta externa. Debido a esta dependencia, PostgreSQL debe ejecutar la subconsulta repetidamente, una vez por cada fila procesada por la consulta principal.

Las subconsultas correlacionadas ofrecen una gran flexibilidad para resolver reglas de negocio complejas; sin embargo, su utilización puede afectar el rendimiento cuando se aplican sobre grandes volúmenes de información.

Por esta razón, resulta recomendable analizar el plan de ejecución mediante la herramienta EXPLAIN PLAN disponible en pgAdmin, con el propósito de verificar que el optimizador de PostgreSQL utilice los índices adecuados y mantenga tiempos de respuesta acordes con los requerimientos de la aplicación.

Caso de uso: El sistema de gestión de recaudo de compañía de transporte (fase de operaciones CRUD)

Al finalizar la jornada laboral, la empresa de transporte requiere generar los reportes de liquidación contable para conocer el rendimiento financiero de las rutas y el nivel de actividad de su talento humano. Con este propósito, el experto en datos desarrolla y ejecuta un conjunto de consultas avanzadas y operaciones de unión relacional que permiten consolidar la información solicitada por la gerencia.

El siguiente bloque de consultas integra operaciones de JOIN, funciones de agregación, agrupamiento y paginación para generar reportes consolidados y optimizar la recuperación de información en PostgreSQL:

Transcripción de código. Operaciones JOIN, funciones de agregación, agrupamiento y paginación

```
-- 1. Reporte Consolidado de Recaudo por Conductor (Uso de INNER JOIN y Agrupamientos)

-- Permite conocer cuánto dinero ha recaudado cada conductor registrado que ha operado turnos

SELECT

    c.cedula,

    c.nombre_completo,

    COUNT(t.id_turno) AS total_turnos_operados,

    SUM(t.valor_base_recaudo) AS total_dinero_recaudado,

    ROUND(AVG(t.valor_base_recaudo), 2) AS promedio_por_turno

FROM conductores c

INNER JOIN asignacion_turnos t ON c.cedula = t.cedula_conductor

GROUP BY c.cedula, c.nombre_completo

HAVING SUM(t.valor_base_recaudo) > 100000.00
```

```
ORDER BY total_dinero_recaudado DESC;

 -- 2. Auditoría Preventiva de la Flota (Uso de LEFT JOIN para detectar inactividad)

-- Permite identificar de forma inmediata qué vehículos no registran ningún turno asignado hoy

SELECT

    v.placa,

    v.modelo,

    t.id_turno,

    t.fecha_turno

FROM vehiculos v

LEFT JOIN asignacion_turnos t ON v.placa = t.placa_vehiculo AND t.fecha_turno = CURRENT_DATE

WHERE t.id_turno IS NULL;

 -- 3. Consulta Paginada de Control Operativo para el Backend

-- Entrega los despachos del día en bloques de 10 registros para optimizar el consumo de la APP

SELECT t.id_turno, t.placa_vehiculo, t.valor_base_recaudo

FROM asignacion_turnos t

WHERE t.fecha_turno = CURRENT_DATE
```

```
ORDER BY t.id_turno ASC

LIMIT 10 OFFSET 0; -- Bloque o página 1
```

Una vez ejecutadas las consultas, es posible analizar el comportamiento del motor PostgreSQL y comprender la forma en que las cláusulas de agrupamiento y las operaciones de unión contribuyen a la generación de reportes gerenciales. El siguiente análisis explica el comportamiento de las consultas implementadas y su aporte al procesamiento de la información:

- **Precisión del control de grupos.** Al implementar la condición HAVING SUM(t.valor_base_recaudo) > 100000.00 en el primer reporte, PostgreSQL calcula inicialmente el valor total recaudado por cada conductor y, posteriormente, excluye aquellos grupos que no superan el umbral financiero establecido. Este comportamiento evidencia la diferencia funcional entre HAVING, que actúa después del agrupamiento, y WHERE, que filtra los registros antes de ejecutar GROUP BY.
- **Detección de ausencias técnicas.** El segundo reporte emplea un LEFT JOIN para identificar los vehículos sin actividad durante la jornada. Al aplicar la condición WHERE t.id_turno IS NULL, el sistema selecciona únicamente los registros del catálogo de vehículos que no presentan coincidencias en la relación de asignaciones del día, permitiendo identificar los autobuses que permanecieron inactivos y que, por tanto, no generaron ingresos para la empresa.

El uso adecuado de las subconsultas, las funciones de agregación y las operaciones JOIN permite construir consultas de alto nivel capaces de consolidar información distribuida y transformarla en indicadores útiles para la toma de decisiones dentro de la organización.

3. Lógica programable en el servidor (PL/pgSQL)

El desarrollo de soluciones informáticas modernas exige que las reglas críticas del negocio no dependan exclusivamente de las capas de aplicación externas, como el frontend o el backend. Centralizar la lógica transaccional en el servidor reduce el tráfico de red, disminuye la latencia y fortalece la consistencia de los datos durante la ejecución de las operaciones. Con este propósito, PostgreSQL incorpora un entorno de programación que permite implementar procesos, automatizar tareas y ejecutar reglas de negocio directamente en el motor de base de datos.

3.1. Programación procedural en PostgreSQL

El lenguaje SQL tradicional responde a un paradigma declarativo, en el cual el desarrollador especifica qué información desea consultar o modificar, sin definir de manera explícita el flujo de ejecución necesario para lograrlo.

Para incorporar estructuras propias de la programación imperativa, como variables, condicionales y ciclos, PostgreSQL implementa PL/pgSQL (Procedural Language/PostgreSQL), un entorno de programación que se ejecuta directamente en el servidor dentro de un espacio de memoria aislado.

En este apartado se estudiará la sintaxis básica para declarar variables y controlar el flujo de ejecución. Asimismo, se analizarán las diferencias funcionales entre las funciones almacenadas, orientadas al retorno de datos, y los procedimientos almacenados, diseñados para administrar procesos transaccionales.

Estructuras de control de flujo, ciclos, condicionales y declaración de variables

Un bloque de código desarrollado en PL/pgSQL posee una estructura jerárquica claramente definida, organizada mediante palabras clave que delimitan cada una de sus secciones. A continuación, se presentan los componentes fundamentales de un bloque procedural:

- **DECLARE**. Define las variables locales, sus identificadores y los tipos de datos que serán utilizados durante la ejecución del bloque.
- **BEGIN**. Contiene las instrucciones que conforman la lógica del programa.
- **END**. Finaliza el bloque de código y marca el cierre de la ejecución.

Para implementar la lógica de negocio, PL/pgSQL incorpora diferentes estructuras de control que permiten tomar decisiones y repetir procesos. A continuación, se resumen las principales estructuras utilizadas durante el desarrollo procedural.

Tabla 1. Principales estructuras de control de flujo en PL/pgSQL

Estructura	Aplicación
IF...THEN...ELSE...END IF	Ejecuta acciones diferentes según el cumplimiento de una condición lógica.
CASE	Evalúa múltiples condiciones y selecciona una alternativa de ejecución.
LOOP	Repite un conjunto de instrucciones hasta que se cumpla una condición de salida.
WHILE	Ejecuta un bloque de instrucciones mientras una condición permanezca verdadera.
FOR registro IN SELECT... LOOP	Recorre automáticamente los registros devueltos por una consulta, permitiendo procesar colecciones relacionales de forma eficiente mediante cursores implícitos.

El uso adecuado de estas estructuras permite implementar procesos automatizados y reglas de negocio directamente en el servidor, reduciendo la necesidad de trasladar el procesamiento a las aplicaciones cliente.

Creación, modificación y ejecución de funciones personalizadas (Stored Functions)

Las funciones almacenadas se crean mediante la sentencia CREATE FUNCTION y se fundamentan en un modelo de procesamiento que recibe parámetros de entrada, ejecuta una secuencia de instrucciones y retorna obligatoriamente un valor o un conjunto estructurado de registros.

Uno de los aspectos técnicos más importantes durante su definición corresponde a la especificación de la volatilidad de la función, característica que permite al optimizador determinar si el resultado puede reutilizarse para mejorar el rendimiento de las consultas. A continuación, se presentan los niveles de volatilidad disponibles en PostgreSQL:

- **IMMUTABLE.** Siempre produce el mismo resultado para los mismos parámetros de entrada. PostgreSQL puede reutilizar el resultado y optimizar su ejecución mediante almacenamiento en memoria.
- **STABLE.** Mantiene un resultado constante durante la ejecución de una consulta, aunque puede cambiar entre diferentes transacciones.
- **VOLATILE.** Puede producir resultados distintos incluso dentro de una misma consulta, por lo que el optimizador debe ejecutarla cada vez que sea invocada.

Las funciones almacenadas pueden utilizarse directamente dentro de sentencias SELECT, WHERE, INSERT, UPDATE y otras instrucciones SQL, lo que facilita la incorporación de lógica reutilizable en las consultas.

La siguiente función ejemplifica la creación de una función almacenada en PL/pgSQL, la cual recibe un parámetro, realiza un procesamiento y retorna un valor al finalizar su ejecución:

Transcripción de código. Sintaxis base de una función en PL/pgSQL

```
CREATE OR REPLACE FUNCTION calcular_antiguedad(anio_modelo INT)
RETURNS INT AS $$
BEGIN
    RETURN EXTRACT(YEAR FROM CURRENT_DATE) - anio_modelo;
END;
$$ LANGUAGE plpgsql STABLE;
```

Desarrollo de procedimientos almacenados (Stored Procedures)

Los procedimientos almacenados fueron incorporados en las versiones modernas de PostgreSQL para ofrecer una implementación alineada con el estándar ANSI SQL. Se crean mediante la sentencia CREATE PROCEDURE y, a diferencia de las funciones, no están orientados al retorno de valores, sino a la ejecución de procesos. Su invocación se realiza mediante el comando CALL.

La principal ventaja de los procedimientos almacenados consiste en la capacidad de administrar el control transaccional desde el propio servidor, permitiendo confirmar o revertir cambios durante la ejecución de un proceso. A continuación, se presentan las principales diferencias entre funciones y procedimientos almacenados.

Tabla 2. Diferencias entre funciones almacenadas y procedimientos almacenados en PostgreSQL

Funciones almacenadas	Procedimientos almacenados
Se crean mediante CREATE FUNCTION.	Se crean mediante CREATE PROCEDURE.
Deben retornar un valor o un conjunto de registros.	No requieren retornar valores.
Pueden utilizarse dentro de consultas SQL.	Se ejecutan mediante el comando CALL.
Están orientadas a encapsular lógica reutilizable y operaciones de consulta.	Están orientados a procesos administrativos y operaciones transaccionales complejas.
No administran el control transaccional mediante COMMIT o ROLLBACK de la misma forma que los procedimientos.	Permiten ejecutar COMMIT y ROLLBACK durante el procesamiento, facilitando la ejecución de procesos por lotes y operaciones de larga duración.

La administración del control transaccional convierte a los procedimientos almacenados en una herramienta esencial para procesos como cierres contables, sincronizaciones masivas de inventarios o cualquier operación que requiera procesar grandes volúmenes de información de manera segura y controlada.

3.2. Control de errores y robustez del código

Las aplicaciones empresariales deben garantizar que los errores durante la ejecución de procesos no comprometan la integridad de la información ni interrumpan innecesariamente las operaciones del sistema. Para ello, PL/pgSQL incorpora mecanismos que permiten detectar excepciones, administrar fallos de forma controlada

y preservar la consistencia de los datos durante la ejecución de procesos transaccionales.

Gestión y captura avanzada de excepciones técnicas (EXCEPTION)

PL/pgSQL incorpora un subsistema de captura de errores mediante la cláusula EXCEPTION, ubicada inmediatamente antes del cierre del bloque principal de ejecución. Cuando se produce un error dentro del bloque BEGIN, PostgreSQL interrumpe la instrucción en ejecución y transfiere automáticamente el control hacia la sección destinada al tratamiento de excepciones, evitando que el fallo sea propagado directamente a la aplicación cliente. El lenguaje permite interceptar diferentes tipos de errores mediante identificadores estándar del sistema.

- **UNIQUE_VIOLATION**. Detecta intentos de insertar registros con valores duplicados en restricciones de unicidad o claves primarias.
- **FOREIGN_KEY_VIOLATION**. Identifica violaciones de integridad referencial ocasionadas por claves foráneas sin correspondencia.
- **DIVISION_BY_ZERO**. Captura operaciones aritméticas que intentan dividir un valor entre cero.

Durante el procesamiento de una excepción, es posible acceder a información de diagnóstico suministrada por PostgreSQL, como el código del error (SQLSTATE) y el mensaje descriptivo (SQLERRM). Estos elementos permiten implementar respuestas específicas según el tipo de incidente detectado.

Uso de bloques condicionales para resguardar la consistencia de los datos en ejecuciones por lotes

Como complemento del manejo de excepciones, PL/pgSQL incorpora la instrucción RAISE, la cual permite emitir mensajes personalizados hacia la consola administrativa, registrar advertencias en la bitácora del servidor o provocar deliberadamente la interrupción de una transacción mediante RAISE EXCEPTION. La combinación de bloques condicionales, subbloques anidados y mecanismos de captura de errores permite preservar la consistencia atómica de los datos durante procesos masivos.

Cuando un registro incumple una regla de negocio, el bloque de control puede capturar el incidente, descartar únicamente la operación afectada o registrar el evento en una relación de auditoría, permitiendo que el procesamiento de los demás registros continúe sin afectar la ejecución global del proceso.

Caso de uso. Sistema de gestión de recaudo de la empresa de transporte (fase de lógica procedural)

La empresa de transporte requiere automatizar el proceso de liquidación financiera diaria de los turnos de los conductores, garantizando que el cálculo incorpore las penalizaciones correspondientes por antigüedad de los vehículos y que el procedimiento continúe operando de manera controlada ante posibles errores de inserción.

Con este propósito, el experto en datos diseña e implementa el siguiente procedimiento almacenado en la base de datos corporativa. El siguiente procedimiento almacenado ejemplifica la automatización de una regla de negocio, incorporando validaciones, control transaccional y manejo de excepciones mediante PL/pgSQL:

Transcripción de código. Creación del procedimiento almacenado para la liquidación diaria transaccional

```
CREATE OR REPLACE PROCEDURE liquidar_jornada_conductor(
    p_cedula VARCHAR,
    p_placa VARCHAR,
    p_valor_bruto NUMERIC
)
AS $$
DECLARE
    v_modelo_vehiculo INT;
    v_antiguedad INT;
    v_descuento_mantenimiento NUMERIC := 0;
    v_valor_final_recaudo NUMERIC;
BEGIN
    -- 1. Extracción y validación de datos base del catálogo físico
    SELECT modelo INTO v_modelo_vehiculo
    FROM vehiculos
    WHERE placa = p_placa;
```

```
    -- Invocación de la función estable para determinar la antigüedad
    v_antiguedad := calcular_antiguedad(v_modelo_vehiculo);

    -- 2. Aplicación de regla de negocio: Penalización por desgaste de flota vieja
    IF v_antiguedad > 5 THEN
        v_descuento_mantenimiento := p_valor_bruto * 0.05; -- 5% de descuento técnico
    END IF;
        v_valor_final_recaudo := p_valor_bruto - v_descuento_mantenimiento;

    -- 3. Intento de inserción transaccional en la tabla operativa
    INSERT INTO asignacion_turnos (placa_vehiculo, cedula_conductor, valor_base_recaudo)
    VALUES (p_placa, p_cedula, v_valor_final_recaudo);

    -- Confirmación explícita de los cambios si no hay errores
    COMMIT;
    RAISE NOTICE 'Liquidación procesada con éxito para el conductor %', p_cedula;
```

```
EXCEPTION
    -- 4. Bloque defensivo de captura de excepciones
    WHEN UNIQUE_VIOLATION THEN
        -- Evita duplicar el turno del conductor para la misma fecha
        ROLLBACK;
        RAISE WARNING 'Excepción controlada: El conductor % ya posee un turno registrado hoy.', p_cedula;

    WHEN OTHERS THEN
        -- Captura cualquier otro error físico imprevisto
        ROLLBACK;
        RAISE EXCEPTION 'Fallo crítico en el servidor: % - %', SQLSTATE, SQLERRM;
END;
$$ LANGUAGE plpgsql;
```

Una vez implementado el procedimiento, es posible analizar el comportamiento del motor PostgreSQL y la manera en que la lógica procedural fortalece la consistencia y la confiabilidad del proceso transaccional.

- **Centralización de la regla de negocio.** El cálculo de la penalización del 5 % aplicable a los autobuses con más de cinco años de servicio se ejecuta directamente en la memoria compartida del servidor PostgreSQL. La aplicación backend únicamente invoca el procedimiento mediante el comando CALL liquidar_jornada_conductor(...), reduciendo la transferencia de datos y garantizando que la lógica financiera permanezca centralizada en el motor de base de datos.
- **Robustez y control atómico.** Al implementar el manejo específico de la excepción WHEN UNIQUE_VIOLATION, si un operador registra accidentalmente la misma operación en dos ocasiones, PostgreSQL intercepta el error de duplicidad de clave, ejecuta un ROLLBACK seguro sobre la operación afectada y genera un mensaje de advertencia controlado. De esta manera, se evita la interrupción del proceso y se preserva la estabilidad de la aplicación utilizada por la empresa.

El uso conjunto de EXCEPTION, RAISE, bloques condicionales y procedimientos almacenados permite desarrollar soluciones transaccionales robustas, capaces de responder adecuadamente ante situaciones imprevistas y de mantener la integridad de la información durante la ejecución de procesos críticos.

4. Automatización reactiva y estructuras de abstracción de datos

El diseño de infraestructuras modernas de bases de datos no solo requiere componentes lógicos ejecutados bajo demanda por las aplicaciones, sino también mecanismos autónomos y reactivos capaces de auditar y hacer cumplir las reglas del negocio de manera inmediata frente a cualquier modificación de la información.

De igual forma, a medida que los repositorios relacionales incrementan su volumen transaccional, resulta indispensable incorporar capas de abstracción que simplifiquen las consultas complejas y estructuras que mejoren el rendimiento del servidor.

4.1. Automatización del negocio mediante disparadores

En las soluciones informáticas empresariales existen procesos que deben ejecutarse obligatoriamente como consecuencia de una operación de modificación de datos. Por ejemplo, registrar una venta puede implicar la actualización automática del inventario, mientras que modificar el salario de un empleado puede requerir el almacenamiento de un registro histórico en una relación de auditoría.

Delegar estas tareas a las aplicaciones cliente incrementa el riesgo de inconsistencias cuando una regla de negocio no es implementada correctamente. Los Triggers eliminan este riesgo al ejecutar dichas acciones directamente en el servidor, de manera transparente para las aplicaciones externas.

Arquitectura, ámbitos y ciclo de vida de un disparador (Triggers)

Un Trigger es un objeto asociado exclusivamente a una relación o a una vista que se ejecuta automáticamente cuando ocurre un evento del Lenguaje de Manipulación de Datos (DML), como INSERT, UPDATE o DELETE.

En PostgreSQL, la implementación de un disparador requiere la creación de dos componentes independientes mediante instrucciones del Lenguaje de Definición de Datos (DDL).

- **Función de trigger (Trigger Function).** Función desarrollada en PL/pgSQL que no recibe parámetros convencionales y retorna obligatoriamente el tipo especial TRIGGER. Contiene la lógica que será ejecutada cuando ocurra el evento asociado.
- **Disparador (Trigger Object).** Objeto creado mediante la sentencia CREATE TRIGGER, encargado de asociar la función previamente compilada con una relación específica y definir el evento que activará su ejecución.

El comportamiento de un disparador también puede configurarse según la granularidad con la que debe ejecutarse:

- **FOR EACH ROW.** Ejecuta el disparador una vez por cada tupla afectada por la operación DML. Es la modalidad más utilizada para procesos de validación, auditoría y transformación de datos.
- **FOR EACH STATEMENT.** Ejecuta el disparador una única vez por cada instrucción SQL, independientemente del número de filas afectadas. Resulta apropiada para registrar eventos generales o actividades administrativas.

Comprender la arquitectura y el ámbito de ejecución de un disparador permite seleccionar la estrategia más adecuada según el tipo de proceso que se desea automatizar.

Creación de funciones de trigger y su asociación a eventos DML (BEFORE, AFTER, INSTEAD OF)

El momento en que un disparador se ejecuta dentro del ciclo de vida de una transacción determina el efecto que tendrá sobre la operación realizada por el motor PostgreSQL. A continuación, se presentan las modalidades de activación disponibles.

- **BEFORE.** Se ejecuta antes de que PostgreSQL valide las restricciones de integridad y escriba los cambios en el almacenamiento físico. Permite modificar los valores de la tupla entrante o cancelar la operación antes de que se realicen cambios permanentes.
- **AFTER.** Se ejecuta una vez que la operación DML ha finalizado correctamente. Resulta adecuado para registrar auditorías, actualizar información relacionada o replicar datos hacia otras relaciones, ya que la operación principal fue validada exitosamente.
- **INSTEAD OF.** Se utiliza exclusivamente sobre vistas que no admiten modificaciones directas. Permite redirigir las operaciones DML hacia las relaciones base correspondientes.

Cuando un disparador se configura con la modalidad FOR EACH ROW, PL/pgSQL proporciona automáticamente variables especiales que representan el estado de la información durante la transacción.

Contiene la nueva versión de la tupla durante operaciones INSERT y UPDATE.

Contiene la versión anterior de la tupla durante operaciones UPDATE y DELETE.

El uso adecuado de los eventos BEFORE, AFTER e INSTEAD OF, junto con las variables NEW y; permite implementar mecanismos de validación, auditoría y

automatización que fortalecen la integridad y la consistencia de la información administrada por PostgreSQL.

4.2. Abstracción lógica y optimización de consultas

A medida que el modelo físico de una base de datos se normaliza, las consultas utilizadas por los módulos de reportería y las aplicaciones se vuelven más complejas, ya que incorporan múltiples operaciones JOIN, subconsultas y funciones de agregación. Repetir estas estructuras en diferentes componentes del software incrementa la probabilidad de errores y dificulta el mantenimiento cuando el esquema de la base de datos evoluciona.

Para reducir esta complejidad, PostgreSQL incorpora mecanismos de abstracción lógica y optimización que permiten simplificar el acceso a la información y mejorar el rendimiento de las consultas.

Vistas estándar como mecanismos de simplificación y seguridad de consultas para el usuario final

Una vista estándar, creada mediante la sentencia CREATE VIEW, es un objeto lógico respaldado por una consulta SELECT almacenada en el catálogo del sistema. A diferencia de una relación física, no almacena información duplicada, sino que genera dinámicamente los resultados cada vez que es consultada (Elmasri y Navathe, 2021).

Cuando una aplicación ejecuta una consulta sobre una vista, PostgreSQL combina automáticamente la definición almacenada de la vista con la consulta solicitada y obtiene la información directamente desde las relaciones base. Además de simplificar el desarrollo de aplicaciones, las vistas constituyen un mecanismo importante para el control de acceso a la información.

- **Simplificación de consultas.** Encapsulan consultas complejas bajo un único identificador, reduciendo la repetición de código y facilitando el mantenimiento del sistema.
- **Control de acceso.** Permiten conceder permisos sobre información específica sin exponer directamente las relaciones que contienen datos sensibles. Por ejemplo, es posible autorizar únicamente la consulta de información pública y restringir campos relacionados con salarios o datos personales.

El uso de vistas facilita el desarrollo de aplicaciones y fortalece la seguridad al limitar el acceso únicamente a la información necesaria para cada usuario.

Vistas materializadas y su impacto en el rendimiento de reportes y consultas complejas

Cuando una consulta procesa grandes volúmenes de información histórica y realiza operaciones de agregación complejas, ejecutar continuamente una vista estándar puede afectar el rendimiento del servidor, debido a que PostgreSQL debe recalcular el resultado en cada solicitud. Para atender este tipo de escenarios, PostgreSQL incorpora las vistas materializadas, creadas mediante la sentencia CREATE MATERIALIZED VIEW. A continuación, se presentan las principales diferencias entre ambos mecanismos:

- **Genera resultados en tiempo real.** Genera los resultados en tiempo real a partir de las relaciones base. Almacena físicamente el resultado de la consulta en el almacenamiento del servidor.

- **Muestra siempre datos actualizados.** Siempre refleja la información más reciente disponible. Representa una instantánea de los datos existentes en el momento de su actualización.
- **Mayor tiempo de procesamiento.** Puede requerir mayor tiempo de procesamiento cuando las consultas son complejas. Reduce significativamente los tiempos de respuesta en procesos analíticos y de reportería.
- **No requiere actualización manual.** No requiere procesos adicionales de actualización. Debe sincronizarse mediante REFRESH MATERIALIZED VIEW, operación que puede ejecutarse de forma concurrente utilizando la opción CONCURRENTLY cuando las condiciones del sistema lo permiten.

Las vistas materializadas resultan especialmente útiles en procesos analíticos donde la prioridad es obtener respuestas rápidas sobre grandes volúmenes de información y los datos no requieren actualización permanente.

Estructuras de indexación (B-Tree, Hash) para la aceleración del acceso a los datos en producción

Un índice es una estructura auxiliar que PostgreSQL crea y mantiene para acelerar la localización de registros, evitando la ejecución de escaneos secuenciales completos (`Sequential Scan`) sobre las relaciones. El motor ofrece diferentes tipos de índices, cada uno optimizado para un conjunto específico de operaciones.

- **B-Tree.** Es la estructura predeterminada y la más versátil de PostgreSQL. Optimiza consultas que utilizan operadores de comparación (<, <=, =, >=, >) y

búsquedas por rangos. Las restricciones PRIMARY KEY y UNIQUE generan automáticamente este tipo de índice.

- **Hash**. Está diseñado para consultas basadas exclusivamente en comparaciones de igualdad (=). Almacena valores mediante funciones hash, lo que favorece búsquedas puntuales, aunque no resulta adecuado para consultas por rangos ni para operaciones de ordenamiento.

La selección del tipo de índice debe realizarse de acuerdo con el patrón de consultas predominante en la aplicación, buscando un equilibrio entre el rendimiento de lectura y el costo de mantenimiento de las estructuras de indexación.

Caso de uso. Sistema de gestión de recaudo de la empresa de transporte (fase de automatización y abstracción)

La empresa de transporte requiere implementar un mecanismo de auditoría automática que registre de manera permanente cualquier modificación realizada sobre las planillas de recaudo diario de los autobuses. Al mismo tiempo, la gerencia necesita disponer de reportes mensuales con tiempos de respuesta mínimos, sin afectar el rendimiento de la aplicación móvil utilizada por los conductores durante la operación.

Para satisfacer estos requerimientos, el experto en datos diseña e implementa los componentes correspondientes en el servidor PostgreSQL:

Transcripción de código. Creación de componentes
-- 1. Creación de la tabla de auditoría física para el histórico de cambios CREATE TABLE auditoria_recaudo (

```
    id_auditoria SERIAL PRIMARY KEY,
    placa_vehiculo VARCHAR(6),
    valor_anterior NUMERIC(12,2),
    valor_nuevo NUMERIC(12,2),
    usuario_aplicacion TEXT DEFAULT CURRENT_USER,
    fecha_modificacion TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);


-- 2. Codificación de la función de trigger en PL/pgSQL
CREATE OR REPLACE FUNCTION fn_trg_auditar_cambio_recaudo()
RETURNS TRIGGER AS $$
BEGIN
    -- Validación técnica: Solo registrar si el valor financiero realmente cambió
    IF OLD.valor_base_recaudo <> NEW.valor_base_recaudo THEN
        INSERT INTO auditoria_recaudo (placa_vehiculo, valor_anterior, valor_nuevo)
        VALUES (OLD.placa_vehiculo, OLD.valor_base_recaudo,
NEW.valor_base_recaudo);
    END IF;
    RETURN NEW; -- Permite que la operación UPDATE continúe su curso normal
```

```
END;

$$ LANGUAGE plpgsql;



-- 3. Asociación del disparador físico a la tabla operativa de asignación de turnos

CREATE TRIGGER trg_auditar_turnos

AFTER UPDATE ON asignacion_turnos

FOR EACH ROW

EXECUTE FUNCTION fn_trg_auditar_cambio_recaudo();



-- 4. Creación de una Vista Materializada para el reporte ejecutivo de gerencia

CREATE MATERIALIZED VIEW mv_resumen_mensual_recaudo AS

SELECT

    v.placa,

    v.modelo,

    COUNT(t.id_turno) AS total_despachos,

    SUM(t.valor_base_recaudo) AS recaudo_acumulado

FROM vehiculos v

INNER JOIN asignacion_turnos t ON v.placa = t.placa_vehiculo
```

```
GROUP BY v.placa, v.modelo

ORDER BY recaudo_acumulado DESC;



-- 5. Creación de un Índice explícito para acelerar consultas por modelo de flota

CREATE INDEX idx_vehiculos_modelo ON vehiculos USING btree (modelo);
```

Una vez implementados los componentes, es posible analizar el comportamiento del sistema y la contribución de cada mecanismo al fortalecimiento de la seguridad y del rendimiento.

- **Garantía de reactividad y no repudio.** Cuando un usuario ejecuta una sentencia UPDATE sobre la relación asignacion_turnos, el disparador trg_auditar_turnos se activa automáticamente después de que la modificación ha sido confirmada (AFTER). Durante la ejecución, la variable OLD conserva el valor original de la operación y NEW registra el valor actualizado. Ambos datos, junto con el usuario obtenido mediante CURRENT_USER, se almacenan en la relación de auditoría, fortaleciendo la trazabilidad y reduciendo el riesgo de alteraciones no autorizadas.
- **Aceleración de reportes mediante almacenamiento físico.** La gerencia puede consultar la vista materializada mv_resumen_mensual_recaudo sin ejecutar nuevamente las operaciones de agregación sobre las relaciones operativas. PostgreSQL recupera directamente la información almacenada, reduciendo considerablemente los tiempos de respuesta. Al finalizar la

jornada, la ejecución de REFRESH MATERIALIZED VIEW mv_resumen_mensual_recaudo sincroniza la información para mantener actualizados los reportes sin afectar el desempeño del entorno de producción.

La combinación de disparadores, vistas, vistas materializadas e índices constituye una estrategia fundamental para fortalecer la automatización, la seguridad y el rendimiento de las bases de datos PostgreSQL utilizadas en entornos empresariales.

5. Perímetros de seguridad, control de acceso granular y continuidad del negocio

Una infraestructura de datos relacional destinada a entornos de producción debe incorporar mecanismos que protejan la confidencialidad de la información y garanticen la continuidad operativa frente a fallos físicos o lógicos. En este contexto, el especialista en bases de datos debe considerar el servidor como un perímetro expuesto a amenazas constantes, lo que hace indispensable implementar controles de acceso, mecanismos de autenticación y estrategias de recuperación de la información.

5.1. Directivas de seguridad perimetral y autenticación en el servidor

El primer nivel de protección de PostgreSQL se encuentra en la capa de red del sistema operativo. Antes de validar los privilegios de un usuario sobre la información almacenada, el servidor debe comprobar el origen de la conexión y el mecanismo utilizado para autenticar la identidad del cliente.

Configuración de directivas de escucha global (postgresql.conf)

Después de una instalación estándar en un entorno local, PostgreSQL configura el proceso principal para aceptar conexiones únicamente desde la interfaz de bucle local (localhost o 127.0.0.1). Cuando las aplicaciones se ejecutan en servidores remotos o en infraestructuras distribuidas, el administrador debe modificar la directiva listen_addresses dentro del archivo postgresql.conf para permitir conexiones externas. A continuación, se presentan las principales directivas relacionadas con la configuración de la escucha del servidor;

- **listen_addresses.** Define las interfaces de red desde las cuales PostgreSQL aceptará conexiones. La configuración listen_addresses = '*' habilita la recepción de solicitudes desde todas las interfaces disponibles.
- **ssl = on.** Activa el uso de conexiones cifradas mediante los protocolos SSL/TLS, protegiendo la información transmitida entre el servidor y las aplicaciones cliente.

Aunque la configuración listen_addresses = '*' facilita el acceso remoto al servidor, también amplía la superficie de exposición. Por esta razón, debe implementarse junto con mecanismos de autenticación y comunicaciones cifradas que fortalezcan la seguridad de la infraestructura.

Control de acceso perimetral mediante el archivo pg_hba.conf

El archivo pg_hba.conf (Host-Based Authentication) constituye el mecanismo nativo de control de acceso de PostgreSQL. Cada registro define una regla que determina si una conexión será aceptada o rechazada, teniendo en cuenta el origen de la solicitud, el usuario, la base de datos y el método de autenticación configurado. A continuación, se presentan los elementos que conforman cada regla de autenticación.

Tabla 3. Campos que conforman una regla de autenticación en el archivo pg_hba.conf

Campo	Descripción
Tipo de conexión	Define si la conexión se realiza de forma local o mediante TCP/IP.
Base de datos	Especifica la base de datos sobre la cual aplica la regla.
Usuario	Indica el usuario o grupo de usuarios autorizado por la configuración.
Dirección IP / Máscara de red	Determina el origen permitido para establecer la conexión.

Campo	Descripción
Método de autenticación	Define el mecanismo mediante el cual PostgreSQL verificará la identidad del usuario.

El siguiente ejemplo presenta la estructura básica de una regla de autenticación en el archivo pg_hba.conf, utilizada para controlar el acceso de usuarios al servidor PostgreSQL según el origen de la conexión y el método de autenticación:

Transcripción de código. Estructura básica de una regla de autenticación

```
# Formato técnico de una regla de exclusión en pg_hba.conf
# TIPO    BASE_DATOS      USUARIO        DIRECCIÓN_IP         MÉTODO
  host    sana_red_db     app_backend    192.168.10.25/32     scram-sha-256
  host    all             all            0.0.0.0/0            reject
```

Como práctica recomendada, PostgreSQL utiliza el método de autenticación scram-sha-256, el cual implementa un esquema criptográfico de desafío y respuesta que fortalece la protección de las credenciales durante el proceso de autenticación.

Las reglas definidas en pg_hba.conf se procesan de forma secuencial. La primera coincidencia determina si la conexión será autorizada o rechazada; por ello, el orden de las reglas constituye un aspecto fundamental para garantizar un control de acceso seguro y coherente con las políticas de la organización.

5.2. Administración de identidades corporativas y matriz de privilegios privados

En PostgreSQL, los conceptos tradicionales de usuario y grupo se integran bajo una única entidad denominada rol (ROLE). Este modelo simplifica la administración de identidades y facilita la aplicación del Principio del Menor Privilegio, según el cual cada usuario o componente del sistema debe disponer únicamente de los permisos indispensables para desempeñar sus funciones.

Creación de roles, usuarios y herencia jerárquica

La creación de perfiles de acceso se realiza mediante la sentencia CREATE ROLE. Cuando un rol requiere autenticarse de manera independiente en el servidor, debe configurarse con el atributo LOGIN, adquiriendo el comportamiento tradicional de un usuario.

Como práctica recomendada para la administración de bases de datos, se aconseja crear inicialmente roles de asignación que agrupen permisos comunes y, posteriormente, asociar dichos roles a los usuarios operativos de la organización. A continuación, se presentan los principales elementos involucrados en este modelo de administración.

- **CREATE ROLE.** Crea un nuevo rol dentro de PostgreSQL.
- **LOGIN.** Habilita al rol para iniciar sesión en el servidor como usuario independiente.
- **Rol de asignación.** Agrupa privilegios comunes del negocio sin permitir autenticación directa. Facilita la administración centralizada de permisos.

- **INHERIT.** Permite que un usuario herede automáticamente los privilegios asignados al rol del cual forma parte, simplificando la gestión de permisos en organizaciones con múltiples usuarios.

Este modelo favorece una administración más eficiente, ya que los cambios en los privilegios se realizan sobre los roles de asignación y se propagan automáticamente a los usuarios asociados.

El siguiente ejemplo presenta la creación de un rol de grupo y la asignación de un usuario operativo, aplicando el modelo de herencia de privilegios en PostgreSQL:

Transcripción de código. Creación de un rol de grupo y asignación de un usuario operativo
-- Creación de un rol de grupo (Matriz de acceso para auditores) CREATE ROLE rol_auditoria_salud WITH NOSUPERUSER NOCREATEDB INHERIT; -- Creación de un usuario operativo asignado al rol grupal CREATE USER auditor_clinico WITH PASSWORD 'ClaveSegura_2026' INHERIT; GRANT rol_auditoria_salud TO auditor_clinico;

Matriz de seguridad granular (GRANT y REVOKE)

El control de las operaciones que un usuario puede ejecutar sobre los objetos de una base de datos se administra mediante las instrucciones del Lenguaje de Control de

Datos (DCL). A continuación, se presentan los comandos fundamentales para la asignación y revocación de privilegios.

- **GRANT**. Concede privilegios específicos, como SELECT, INSERT, UPDATE y DELETE, sobre relaciones, vistas u otros objetos de la base de datos a uno o varios roles.
- **REVOKE**. Revoca de forma inmediata los privilegios previamente otorgados, impidiendo la ejecución de operaciones no autorizadas.

La administración de privilegios puede realizarse tanto a nivel de esquema como de relación, permitiendo implementar un modelo de seguridad ajustado a las funciones de cada perfil.

Por ejemplo, una aplicación de facturación médica puede requerir permisos INSERT sobre la relación de cobros para registrar nuevas transacciones, mientras que el personal encargado de la recepción de citas únicamente necesita privilegios SELECT para consultar la información. Esta distribución de permisos reduce el riesgo de modificaciones accidentales o acciones maliciosas que comprometan la integridad de los datos y la seguridad de la organización.

5.3. Protocolos de respaldo

La continuidad del negocio y la resiliencia operativa frente a desastres físicos o lógicos, como fallos de energía o corrupción del almacenamiento, dependen de la implementación de mecanismos de recuperación que permitan restaurar la información de manera confiable. En un entorno corporativo, una base de datos solo puede considerarse protegida cuando dispone de estrategias que permitan reconstruir su estado hasta el momento inmediatamente anterior a una contingencia.

Fundamentos operativos del Log de transacciones o Write-Ahead Logging (WAL)

El mecanismo Write-Ahead Logging (WAL) constituye uno de los pilares tecnológicos que garantizan la propiedad de durabilidad del modelo ACID en PostgreSQL. Su principio de funcionamiento establece que ninguna modificación sobre las tuplas físicas se almacena en las relaciones de datos hasta que el cambio ha sido registrado previamente de forma secuencial en los archivos de transacciones WAL, ubicados en el directorio pg_wal.

Cuando ocurre una interrupción inesperada del servidor, PostgreSQL utiliza la información registrada en estos archivos para recuperar automáticamente el estado consistente de la base de datos. A continuación, se presentan las principales funciones del mecanismo WAL.

- **Registro previo de transacciones.** Almacena cada modificación en los archivos WAL antes de actualizar las relaciones físicas de la base de datos.
- **Recuperación automática.** Durante el reinicio del servidor, PostgreSQL analiza los archivos WAL para reconstruir el estado consistente del clúster.
- **Aplicación de transacciones confirmadas.** Ejecuta nuevamente los cambios asociados a transacciones confirmadas mediante COMMIT que aún no habían sido escritos en las relaciones físicas.
- **Reversión de transacciones incompletas.** Descarta las operaciones que quedaron inconclusas mediante un proceso equivalente a ROLLBACK, preservando la integridad de la información.

Este mecanismo garantiza que la base de datos pueda recuperarse de forma consistente incluso cuando la interrupción ocurre durante la ejecución de operaciones críticas.

Respaldos lógicos y restauración mediante comandos de consola (pg_dump, pg_restore)

Además del registro de transacciones, PostgreSQL incorpora utilidades de consola que permiten generar copias de seguridad lógicas sin interrumpir la disponibilidad del servicio. A continuación, se presentan las principales herramientas utilizadas durante las tareas de respaldo y restauración:

- **pg_dump.** Genera una copia de seguridad de una base de datos específica, incluyendo la estructura lógica (DDL) y los datos (DML). Permite crear archivos en formato de texto (.sql) o en formatos binarios comprimidos, como .backup y .tar.
- **pg_dumpall.** Genera un respaldo completo del clúster PostgreSQL, incluyendo todas las bases de datos, los roles, los usuarios y la configuración global de privilegios. Resulta especialmente útil durante procesos de migración o recuperación integral del servidor.
- **pg_restore.** Restaura copias de seguridad generadas en formatos binarios compatibles con pg_dump, reconstruyendo la estructura y los datos de la base de datos.
- **psql.** Ejecuta archivos de respaldo en formato .sql, permitiendo reconstruir el repositorio mediante instrucciones SQL almacenadas en el `script` de recuperación.

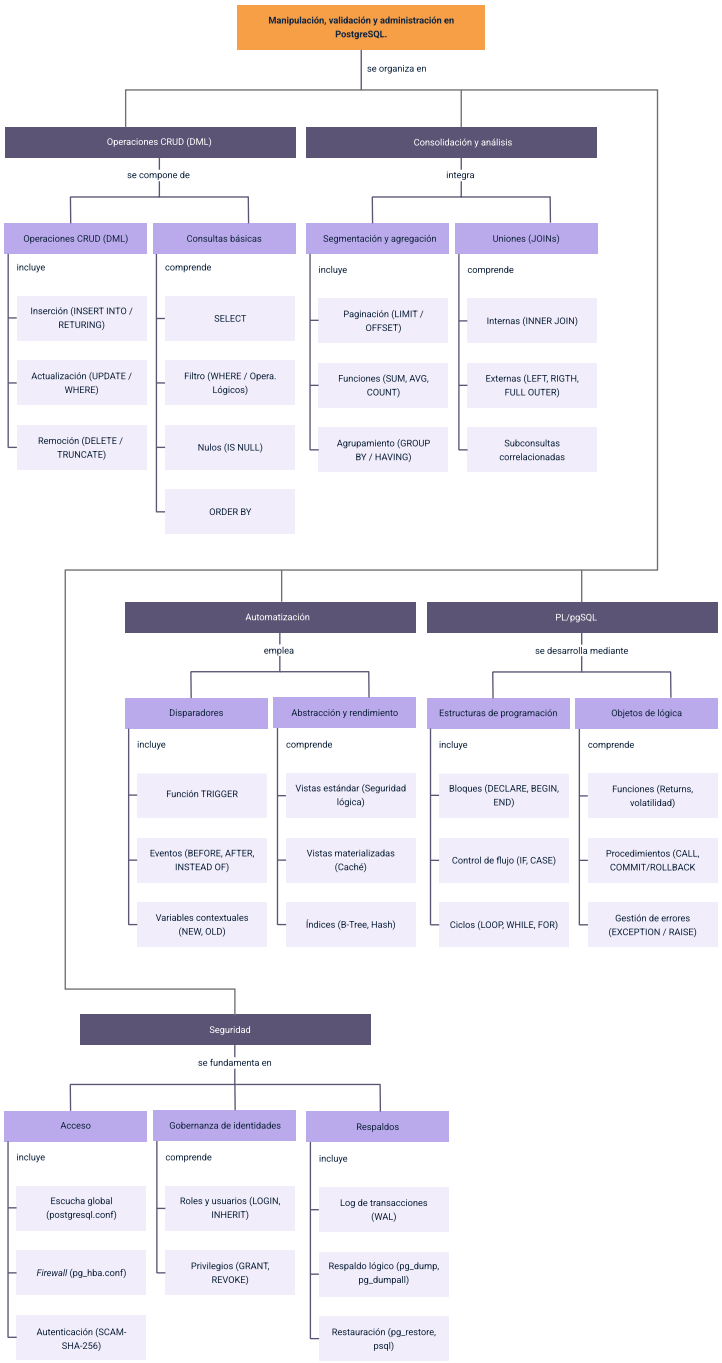
La selección de la herramienta adecuada depende del alcance del respaldo requerido y del formato utilizado para almacenar la copia de seguridad. La combinación de estas utilidades con el mecanismo WAL constituye una estrategia fundamental para garantizar la recuperación de la información y la continuidad operativa de los sistemas basados en PostgreSQL.

El siguiente ejemplo presenta los comandos básicos para generar y restaurar un respaldo lógico de una base de datos PostgreSQL mediante las utilidades pg_dump y pg_restore.

Transcripción de código. Comandos básicos para generar y restaurar un respaldo lógico de una base de datos PostgreSQL.
Comando de consola para realizar un respaldo comprimido de alta fidelidad pg_dump -h host_remoto -U postgres -F c -b -v -f backup_SanaRed_2026.backup sana_red_db # Comando de consola para restaurar el respaldo binario en un nuevo servidor vacío pg_restore -h localhost -U postgres -d sana_red_db -v backup_SanaRed_2026.backup

Síntesis

A continuación, se presenta a manera de síntesis, un esquema que articula los elementos principales abordados en el desarrollo del componente formativo.



Glosario

ACID: acrónimo en inglés de Atomicidad, Consistencia, Aislamiento y Durabilidad (Atomicity, Consistency, Isolation, Durability). Es el conjunto de cuatro propiedades fundamentales que un motor transaccional como PostgreSQL garantiza de forma mandatoria para asegurar que los datos permanezcan válidos e indestructibles ante fallas del sistema.

Backend: capa lógica y de acceso a datos de una aplicación de software que opera del lado del servidor. Se encarga de procesar las reglas de negocio, validar la seguridad y realizar las conexiones operativas directas hacia el motor de base de datos.

CRUD: acrónimo utilizado en la ingeniería de software para condensar las cuatro operaciones esenciales de manipulación de registros en un repositorio físico: Crear (INSERT), Leer (SELECT), Actualizar (UPDATE) y Borrar (DELETE).

DBA: administrador de Bases de Datos (Database Administrator). Rol profesional y técnico encargado de asegurar la correcta infraestructura del motor, gobernar las directivas de seguridad granular, planificar las matrices de permisos, optimizar los índices de rendimiento y garantizar la continuidad del negocio mediante respaldos.

Frontend: capa visual, de presentación e interacción de una solución informática con la que el usuario final interactúa de manera directa (por ejemplo, formularios web o aplicaciones móviles que capturan las solicitudes del negocio).

PL/pgSQL: lenguaje procedural nativo y embebido de PostgreSQL que extiende las capacidades declarativas del estándar SQL, permitiendo estructurar lógica

imperativa mediante variables, control de flujos condicionales, ciclos y captura avanzada de excepciones técnicas.

Triggers: disparadores u objetos de automatización reactiva implementados en el servidor, los cuales ejecutan de forma automática y obligatoria una función procedural inmediatamente antes o después de que se suscite un evento DML específico sobre una tabla.

Tuplas: representación formal dentro del modelo relacional para hacer referencia a una fila, registro o instancia de datos individual alojada dentro de una estructura bidimensional (tabla).

Vistas materializadas: objeto lógico que almacena físicamente en el disco duro el resultado de una consulta relacional pesada, actuando como una instantánea caché de datos para optimizar drásticamente la velocidad de respuesta de los reportes gerenciales de la empresa.

Referencias bibliográficas

Elmasri, R., y Navathe, S. B. (2021). Fundamentals of database systems (8th ed.). Pearson.

Neon. (2026). Separating storage and compute. Neon Docs. https://neon.com/docs/postgres/overview

PostgreSQL Global Development Group. (2026). PostgreSQL 16.0 documentation: Chapter 18. Server setup and operation. https://www.postgresql.org/docs/16/runtime.html

Silberschatz, A., Korth, H. F., y Sudarshan, S. (2002). Fundamentos de bases de datos. McGraw-Hill. https://biblioteca.univalle.edu.ni/files/original/01aebde3cc06dce33f2538aa2724eb2541cb9473.pdf

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional 06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Jaime Hernán Tejada Llano	Experto temático	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Paola Alexandra Moya	Evaluadora instruccional	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Carlos Julian Ramirez Benitez	Diseñador de contenidos digitales	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Jhon Jairo Urueta Alvarez	Desarrollador full stack	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Alejandro Delgado Acosta	Intérprete lenguaje de señas	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Cristhian Giovanni Gordillo Segura	Intérprete Lenguaje de señas	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Juan Pablo Rojas Polania	Animador y productor multimedia	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Carlos Eduardo Garavito Parada	Animador y productor multimedia	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Maria Carolina Tamayo Lopez	Locución	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
German Acosta Ramos	Locución	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Aixa Natalia Sendoya Fernández	Validador de recursos educativos digitales	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila

Nombre	Cargo	Centro de Formación y Regional
Ricardo Oliveros Zambrano	Validador de recursos educativos digitales	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Anyerson Wilfredo Pizo Ossa	Evaluador para contenidos inclusivos y accesibles	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Daniel Ricardo Mutis Gómez	Evaluador para contenidos inclusivos y accesibles	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila